Pythagorean Triples: $a^2+b^2=c^2$
只要有一个解 (a,b,c), 那么对于 positive integer d, $(da)^2+(db)^2=(dc)^2$
Primitive Pythagorean Triple (PPT): 如果 a,b,c 没有公因子就叫 PPT. eg：(3,4,5) 是 PPT；(6,8,10) 不是 PPT—> common factor=2
-soultion of $a^2+b^2=c^2$ must be "odd+even=odd"
Note：看到整数平方要想到奇偶性
Theorem: Every PPT (a,b,c) with a odd satisfies:
$a=st, b=\frac{s^2-t^2}{2}, c=\frac{s^2+t^2}{2}, s>t\geq 1$ are to be odd and no common factor.
整除的基本性质：if d|x, d|y, then d|(x+y),d|(x-y)
Well-ordering Principle: every non-empty set of positive integers contains a least element.
Division Algorithm： Let $a,b\in\mathbb{Z}, b>0$. Then ∃ a unique pair $q,r\in\mathbb{Z}$. s.t. $a=qb+r$; and $0\leq r<b$. q is called the quotient and r is called the remainder.
Def: Let $a,b\in\mathbb{Z}$. if $\exists c\in\mathbb{Z}, s.t. b=ac$, then a divides b, a|b
Note: gcd(0,k)=k; gcd(0,0) is not defined.
Bezout lemma: let a,b be non-zero integers with gcd (a,b). Then ∃ integer s and t s.t. $as+bt=(a,b)$. Any common divisor of a, b divides (a,b) 最大公因数除了 gcd 还能写成 a 和 b 的整数线性组合
eg. a=17, b=5. since gcd(17,5)=1. so ∃ s and t s.t. $17s+5t=1$ -> 1=17-3·5 -> $s=1, t=-3$
Extended Euclidean algorithm: let $a,b\in\mathbb{Z}, \exists integers q_1, r_1$ s.t.
$a=q_1b+r_1$.
$b=q_2r_1+r_2$.
$r_1=q_3r_2+r_3$.
$r_2=q_4r_3+r_4$.
Congruences:
Def: The congruence $a\equiv b \pmod n$ means that the difference $(a-b)$ is divisible by n. In other words, a is equal to a multiple of n plus b, or $a=nq+b$.
eg. 15≡ 1 mod 7, -3≡ 14 mod 17, 10≡ 0 mod 5
Congruences class:
eg. $-4\equiv 1\equiv 6\equiv 11\equiv 16 mod 5$ is a congruence class.
complete system of residue modulo n: 一个集合, 如果每个整数都恰好与其中一个元素同余, 就是 complete system of residue modulo n
eg. modulo 5, complete system of residue is (0,1,2,3,4). Another is (-2,-1,0,1,2) 模几就是几个类：所有 ≡ 1 mod 5 的整数,...≡ 2 mod 5, ...≡ 3 mod 5,...≡ 4 mod 5
Theorem: if a,b,c,d,n are integers, $n>0$. $a\equiv b$mod n, $c\equiv d$mod n then, $a+c\equiv b+d$ mod n; $a-c\equiv b-d$ mod n; $ac\equiv bd$ mod n;
eg. compute 93·17 modulo 6. since $93\equiv 3$ mod 6 and $17\equiv -1$ mod 4. so $93\cdot 17=-3$ mod 6
The ring $\mathbb{Z}_n$:
Def: $\mathbb{Z}_n=\{\overline{0},\overline{1},\dots,\overline{n-1}\}$.
addition and multiplication:
$\overline{a}+\overline{b}=\overline{c}\iff a+b=c \pmod n$.
$\overline{a}-\overline{b}=\overline{c}\iff a-b=c \pmod n$.
$\overline{a}\cdot\overline{b}=\overline{c}\iff a\cdot b=c \pmod n$. where $0\leq a,b<n$.
Wilson's Theorem: if p is prime then $(p-1)!\equiv$ mod p
inverse of $\mathbb{Z}_n$ and application of inverse: 在 $\mathbb{Z}_n$ 中, 如果存在 $\overline{x}$ 使: $\overline{a}\cdot\overline{x}=1, \overline{x}$ 是 $\overline{a}$ 的 inverse —> $ax\equiv 1$ mod n. 什么时候有逆元? a 在 mod n 下有逆元 ⟺ gcd(a,n) =1 (by Bezout identity: if gcd(a,n)=1, ∃ x, y such that $ax+ny=1$ -> $ax\equiv 1$ mod n -> x 有逆元)
eg. what's inverse of 3 mod 7? $3x\equiv 1$ mod7 ->$3\cdot 5=15\equiv 1$ mod 7 -> so $3^{-1}\equiv 5$ mod 7
Chinese reminder theorem:
Def: Let $m_1, m_2, \dots, m_k$ be natural numbers such that the greatest common divisor of any two of them is 1. Let $a_1, a_2, \dots, a_k$ be arbitrary integers. Then there is x such that: $x\equiv a_1 \pmod{m_1}; x\equiv a_2 \pmod{m_2}\dots x\equiv a_k \pmod{m_k}$
This solution (x) is unique modulo $m_1m_2\cdots m_k$.
eg. solve the congruence: $x\equiv 1$ mod 2, $x\equiv 3$ mod 5, $x\equiv 4$ mod 7
solution: $N=2\cdot5\cdot7=70, N_1=\frac{70}{2}=35, N_2=\frac{70}{5}=14, N_3=\frac{70}{7}=10$. So: $35x_1\equiv 1$ mod 2, $14x_2\equiv 1$ mod 5, $10x_3\equiv 1$ mod 7.
We can take $x_1=1, x_2=4, x_3=5$. so
$x=a_1x_1N_1+a_2x_2N_2+a_3x_3N_3=1\cdot1\cdot35+3\cdot4\cdot14+4\cdot5\cdot10=403$ Note: $x=53$ is also a solution as $53=403-70\cdot5$
Theorem: Let m,n be two integers s.t. $gcd(m,n)=1$. Then the congruece $x\equiv a$ mod m·n has the same solution as congruence: $x\equiv a$ mod m , $x\equiv a$ mod n
Conclusion. Knowing a number x mod N = knowing $x$mod each of the prime powers $p_j^{e_j}$ in $N=p_1^{e_1}\cdots p_n^{e_n}$. eg. knowing $x\equiv 27 \pmod{30}$ =knowing $x\equiv 1 \pmod 2$, $x\equiv 0 \pmod 3$, $x\equiv 2 \pmod 5$.
一般原则：对于 $ax\equiv b$ mod n, 先看 $d=gcd(a,n)$, 若 $d\nmid n$, 无解；若 $d\mid n$, 有解, 可以先约掉 d 再求
Binary Code:
Def: A binary code of length n is a subset $C\subseteq\{0,1\}^n$. The elements of C are called codewords. 只有 0 和 1 组成的集合叫 binary code.
eg: $C=\{000,001,010,100\}$ 是一个 binary code of length 3.
Block Code:
Def: A block code have all its codeword the same length
eg. $C=\{000,001,010,100\}$ is a block code of length 3;
$C=\{11111,0000\}$ is not a block code.
Hamming distance:
for all $x=(x_1,\dots,x_n)$, and $y=(y_1,\dots,y_n)$, the Hamming distance $d(x,y)$ is the number of positions at which x and y differ.
$d(x,y)=|\{i\in\{1,\dots,n\}: x_i\neq y_i\}|$
Minimal distance rule:
1. Let F be a field and $C\subseteq F^n$ be a code. A *decision rule* for C is a function $\sigma: F^n\to C$, which assigns to each $z\in F^n$ a codeword $c\in C$.
2. Let C be a code and suppose that a codeword from C was sent through a noisy channel. When a word z is received, the receiver must try to make a reasonable choice as to which codeword $\sigma(z)$ was sent. The *minimal distance rule* says that, given z, the receiver should choose $\sigma(z)$ to be a codeword c such that $d(z,c)$ is minimum. 收到一个 z 后, 选一个 codeword c 使得它和 z 的距离最小—> minimal distance
3.(Theorem:)Suppose that a sender uses a code C having minimum distance $d>2r$, and the receiver uses the minimal distance rule. Then provided that no more than r bit-errors are made in transmitting any codeword, no mistakes will occur: every received word will be restored to the correct codeword. 如果 code MD $d>2r$, 且错误 r 位, 那么 minimal distance rule 能保证正确解码。
Weight:
the weight of x, $wt(x)$ is the number of 1's in x.
$wt(x)=d(x,0)=|\{i\in\{1,\dots,n\}: x_i\neq 0\}|$.
lemma: Let C be a linear code, the minimum distance $d(C)$ of C is equal to the minimum weight of a nonzero codeword in C.
the minimum distance of a code $d(C)$: 两个 codewords 彼此之间能有多远
The minimum distance $d(C)$ of C is defined by setting
$d(C)=\min\{d(x,y): x,y\in C,\ x\neq y\}$.
binary linear code:
Def: A block code that sum of any two codewords is also a codeword is called a linear code. 两个 code 相加等于组里的另一个 code；只要发现一组两个 codeword 相加不在组里, 就不是 linear code 了。
eg. $C=\{000,011,101,110\}$ is a binary linear code.
If C is a binary linear code, then C has 2^k elements for some k

Note: 在这门课, 我们 assume $F=F_2$. 意味着 addition：模 2 加法；multiplication：普通乘法, 但是 **结果为 0/1**
parameters of linear code:
Def: (n,k,d) is a linear code of length n, dimension k, and minimum distance d if C is a linear code with 2^k codewords, each of length n, and minimum distance d (codeword 之间比较).
k：维数 -> $|C|=2^k$, $|C|$ 是 codeword 的数量
eg. F_2^5 -> (5,5,1); $C_1=\{0000,1000,0100,1100\}$ -> (4,2,1)
Generator matrix:
Def: A generator matrix G for a linear code C is a $k\times n$ matrix whose rows form a basis for C,C is a linear code with parameter (n,k,d).
eg. let $C=\{0000,11001,00110,11111\}$ be a linear code. Then generator matrix is $\begin{Bmatrix}1&1&1&1&1\\0&0&1&1&0\end{Bmatrix}$ 分别是 c1 和 c2 的 basis. 任何 codeword 都可以写成 c1 和 c2 的线性组合:
$0c_1+0c_2; 1c_1+0c_2; 0c_1+1c_2; 1c_1+1c_2$, 得到 C 的所有 codeword.
注意：generator matrix 不唯一, 因为 code 的 basis 不唯一。
A Parity Check Matrix:
Orthogonal: $x\cdot y=\sum_{i=1}^n x_iy_i$. Two vectors x, y are orthogonal if $x\cdot y=0$.
这里的 orthogonal 是指 dot product 在 F_2 下等于 0。eg. $1011\cdot1101=0$, 所以 1011 和 1101 是 orthogonal 的。
dual code $C^\perp$: $C^\perp=\{x\in F_2^n: x\cdot c=0 \text{ for all } c\in C\}$
$C^\perp$ 是所有与 C 里面每一个 codeword 都 orthogonal 的 vectors 组成的集合。
self-dual code:A code is called self-dual if $C=C^\perp$. In this case, the code is equal to its dual code.
parity check matrix: A generator matrix for $C^\perp$ is called a parity check matrix for C. which is also called $H, (n-k)\times n$ matrix.
Purpose of parity-check matrix: The parity-check matrix is used to check whether a binary word is a codeword. A binary word c is in C if and only if $Hc^T=0$. Equivalently, $cH^T=0$.
·dim C + dim $C^\perp=n$
lemma: 对于一个 linear code C, 如果 H 是 C 的 parity check matrix, 那么下面等价:
1. C has minimum distance, d
2. H has d columns whose sum is zero vector, and H does not have $d-1$ columns whose sum is zero vector.
Triangle inequality for Hamming distance:
For binary words x, y, z of length n, $d(x,y)+d(y,z)\geq d(x,z)$. This means that going from x to z directly cannot be longer than going from x to y and then from y to z.
Neighborhood :
For a binary word x of length n and $r\geq 0$, the neighborhood of x with radius r is $N_r(x)=\{y\in\{0,1\}^n: d(x,y)\leq r\}$. It contains all binary words whose Hamming distance from x is at most r.
离 code x 不超过 r 位错误的所有 codeword 组成的集合
The size of neighborhood: $|N_r(x)|=\binom{n}{0}+\binom{n}{1}+\cdots+\binom{n}{r}$. n 是单个 codeword 的长度
This is because a word at distance i from x is obtained by choosing i positions to change.
Perfect code: Perfect code 就是 radius-r neighborhoods 既不重叠, 又刚好覆盖全部 2^n 个 binary words.
Def: **if d=2r+1** , 不重叠 + 覆盖整个空间 ⟺ 计数刚好等于 2^n
if $d=2r+1$, Then C is a perfect code if
$|C|\left(\binom{n}{0}+\binom{n}{1}+\cdots+\binom{n}{r}\right)=2^n$.
eg. $C=\{000,111\}$ minimum distance 3.
$d=3=2\cdot r+1->r=1, |C|=2, \binom{3}{0}+\binom{3}{1}=4$, so $|C|(\binom{3}{0}+\binom{3}{1})=8=2^3$.
Euler's function:
defined as: $\varphi(n)=\#\{1\leq a\leq n: gcd(a,n)=1\}$. 从 1 到 n 里, 与 n 互质的数的数量
eg. $\varphi(8)=4$, since 1, 3, 5, 7 are coprime to 8.
Euler's theorem:
Let $n>1$, $gcd(a,n)=1 \implies a^{\varphi(n)}\equiv 1 \pmod n$.
Fermat's little theorem: if p is prime and a is not divisible by p, then $a^{p-1}\equiv 1 \pmod p$. where gcd(a,p)=1.
Theorems:
1. $\gcd(m,n)=1 \implies \varphi(m\cdot n)=\varphi(m)\varphi(n)$
2. $\varphi(p^a)=p^a-p^{a-1}=p^{a-1}(p-1)=p^a\left(1-\frac{1}{p}\right)$.
3. 如果: $n=p_1^{a_1}p_2^{a_2}\cdots p_m^{a_m}$, 其中 $p_1,\dots,p_m$ 是不同的 prime factors, 那么: $\varphi(n)=n\left(1-\frac{1}{p_1}\right)\left(1-\frac{1}{p_2}\right)\cdots\left(1-\frac{1}{p_m}\right)$
4. if $n=p_1^{a_1}p_2^{a_2}\cdots p_m^{a_m}$; then $\varphi(n)=\prod_{i=1}^m p_i^{a_i-1}(p_i-1)$
eg. $120=2^3\cdot3^1\cdot5^1$, so $\varphi(120)=120\left(1-\frac{1}{2}\right)\left(1-\frac{1}{3}\right)\left(1-\frac{1}{5}\right)=32$. 不用管指数是多少
Carmichael number: 如果一个数 n 满足 $a^{n-1}\equiv 1 \pmod n$, 对于所有和 n 互质的 a 都成立. 不一定意味着 n 是 prime.
eg. $561=3\cdot11\cdot17$, it satisfies $a^{560}\equiv 1 \pmod{561}$ for all a coprime to 561, so 561 is a Carmichael number.
eg. $n>1$, find all n with $\varphi(n)=2$
using theorem 4, 1. $p_i-1|2$, 2. $p_i-1\in\{1,2\}$ 3. $p_i\in\{2,3\}$, 4. $n=2^\alpha;3^\alpha;2^\alpha3^\beta$. 5. 带入
$\varphi(p^n)=p^n-p^{n-1}=p^{\alpha-1}(p-1)=p^n(1-\frac{1}{p})$ 6. 得到 a, $n=p^\alpha$
最后 n= 3,4,6
Ring $\mathbb{Z}_n$:
Def: $+: R\times R\to R$ $\cdot: R\times R\to R$
两个元素做完运算后, 结果还是在这个集合里
A1(commutative ring) $a+b=b+a$ A2 $(a+b)+c=a+(b+c)$
A3 $\exists 0_R\in R$ such that $0_R+a=a+0_R=a$ A4 $\forall a\in R, \exists(-a)\in R$ such that $a+(-a)=(-a)+a=0_R$
M1 $a\cdot b=b\cdot a$ M2 $(a\cdot b)\cdot c=a\cdot(b\cdot c)$ M3(unital ring) $\exists 1_R\in R$ such that $1_R\cdot a=a\cdot1_R=a$ D(commutative ring with identity) $a\cdot(b+c)=a\cdot b+a\cdot c,\quad (a+b)\cdot c=a\cdot c+b\cdot c$
Primitive roots:
Def: 设 p 是 prime, $p>0$, 如果一个整数 i 满足 $i^1, i^2, \dots, i^{p-1}$ 模 p 后两两不同, 那么 i 就叫做 prime p 的 primitive root modulo p.
i.e. $i^m\neq 1 \pmod p$ for all $0<m<p-1$.
= Def(1): if j is primitive mod p ⟺ $i^m\not\equiv 1(modp)$ for $0<m<p-1$.
= Def(2): integer i is a primitive root mod p iff $m\equiv i^\alpha modp$ 每一个非零模 p 的整数都可以写成 i^α
Primitive root theorem
Def: For every prime p, there always exists a primitive root mod p.
eg. 2 is not a primitive root mod 7. But 3 is a primitive root mod 7. As $2^1\equiv 2(mod7), 2^2\equiv 4(mod7), 2^3\equiv 1(mod7)$ -> 1,2,4
$3^1\equiv 3(mod7), 3^2\equiv 2(mod7), 3^3\equiv 6(mod7), 3^4\equiv 4(mod7), 3^5\equiv 5(mod7), 3^6\equiv 1(mod7)$ -> 1,2,3,4,5,6 刚好指数是 1 到 p-1, 且结果两两不同且非零 mod 7, 所以 3 是 primitive root mod 7.

orders:
Def: i is primitive root mod p ⟺ $order_p(i)=p-1$.
eg. $2^3\equiv 1 \pmod 7$, so $ord_7(2)=3$, since $3\neq 6$ so 2 is not a primitive root mod 7. $3^6\equiv 1 \pmod 7$, so $ord_7(3)=6$, since $6=6$ so 3 is a primitive root mod 7.
Bezout lemma 应用 if $x^\alpha\equiv 1 \pmod p$ and $x^\beta\equiv 1 \pmod p$, then $x^{\gcd(\alpha,\beta)}\equiv 1 \pmod p$, 用 bezout lemma: ∃$u, v$ integers s.t. $u\alpha+v\beta=\gcd(\alpha,\beta)$ 证明
The legendre symbol:
Def: $p>0$, is a prime, a is an integer not divisible by p. The Legendre symbol is defined as: $\left(\frac{a}{p}\right)$ **注意：** $p\nmid a$
$\left(\frac{a}{p}\right)=\begin{cases}1, & \text{if } a\equiv r^2 \pmod p \text{ for some integer } r,\\ -1, & \text{otherwise.}\end{cases}$
$if \left(\frac{a}{p}\right)=1 \implies$ a is a quadratic residue modulo p.
qudratic residue:
a 在 modulo p 下可以写成某个数的平方；$x^2\equiv a \pmod p$
eg. 我们计算所有非零平方：
$1^2\equiv 1,\quad 2^2\equiv 4,\quad 3^2\equiv 2,\quad 4^2\equiv 2,\quad 5^2\equiv 4,\quad 6^2\equiv 1 \pmod 7$. 所以 modulo 7 下的 quadratic residues 是 1, 2, 4. 因此:
$\left(\frac{1}{7}\right)=\left(\frac{2}{7}\right)=\left(\frac{4}{7}\right)=1$. 而 3, 5, 6 不是任何数的平方，所以:
$\left(\frac{3}{7}\right)=\left(\frac{5}{7}\right)=\left(\frac{6}{7}\right)=-1$.
步骤是: **1.** 计算 $i^1, i^2, \dots, i^{p-1}$ 模 p; **2.** 看是否提前出现 1; **3,** 如果在 $p-1$ 之前出现 1, 就不是 primitive root; **4,** 如果直到 $p-1$ 才出现 1, 就是 primitive root
Theorem: if g is a primitive root mod p, then the quadratic residues mod p are exactly g^α. if $\alpha=2k$ → quadratic residue; if $\alpha=2k+1$ → non-quadratic residue
Legendre symbol- 5 rules:
1. $\left(\frac{a}{p}\right)\equiv a^{\frac{p-1}{2}} \pmod p$, **p must be odd prime**; if a is quadratic residue mod p, then $a^{\frac{p-1}{2}}\equiv 1$ mod p; if a is non-quadratic residue mod p, then $a^{\frac{p-1}{2}}\equiv -1$ mod p.
2. $\left(\frac{ab}{p}\right)=\left(\frac{a}{p}\right)\left(\frac{b}{p}\right)$;
3. $\left(\frac{a}{p}\right)=\left(\frac{a-p}{p}\right)$, if $a\equiv b \pmod p$, then $\left(\frac{a}{p}\right)=\left(\frac{b}{p}\right)$
4. if $p\equiv 1 \pmod 4$ and $q\equiv 1 \pmod 4$, then $\left(\frac{p}{q}\right)=\left(\frac{q}{p}\right)$
5. if $p\equiv 3 \pmod 4$ and $q\equiv 3 \pmod 4$, then $\left(\frac{p}{q}\right)=-\left(\frac{q}{p}\right)$
特殊结论: $\left(\frac{-1}{p}\right)=1$ provided that $p\equiv 1 \pmod 4$, and $\left(\frac{-1}{p}\right)=-1$ provided that $p\equiv 3 \pmod 4$.
解题步骤: **I**. 先把分子化简 mod p, 用 rule 3; **II**. 如果分子可以拆开成几个数的积，用 rule 2; **III**. 遇到 -1, 用特殊结论; **IV**. 如果上下都是 odd primes，用 rule 4 和 rule 5; **V**. 最后一定要化到最小 symbol, 例如 $\left(\frac{2}{7}\right), \left(\frac{1}{p}\right)=1, \left(\frac{-1}{p}\right)=-1$
Gauss Lemma:
Def: Let p be an odd prime and $a\nmid p$. build the set of integers: $S=\{a, 2a, 3a, \dots, \frac{p-1}{2}a\}$, S 中每个元素除以 P 后的 reminder 组成 S', → all elements of S' are chosen to lie in the interval $(-\frac{p}{2}, \frac{p}{2})$.
$\left(\frac{a}{p}\right)=(-1)^n$ n: 在 S' 中小于 0 的元素的数量.
可以直接用: $\left(\frac{2}{p}\right)=(-1)^{\frac{p^2-1}{8}}, p>2 prime. \iff$
$\left(\frac{2}{p}\right)=\begin{cases}1, & p\equiv 1,7 \pmod 8,\\ -1, & p\equiv 3,5 \pmod 8.\end{cases}$
Gaussian integers:
Def: $\mathbb{Z}[i]=\{a+ib: a,b\in\mathbb{Z}\}$. a,b 都是整数
unit in $\mathbb{Z}[i]$:
Def: z is a unit in $\mathbb{Z}[i]$ if there exists $z'\in\mathbb{Z}[i]$ such that $zz'=1$. 证明方法: $zz'=1\longrightarrow|z||z'|=1\longrightarrow|z|=|z'|=1\longrightarrow a^2+b^2=1\longrightarrow a=\pm1, b=0$ 或 $a=0, b=\pm1$
在 $\mathbb{Z}[i]$ 中, units 是 $\{1,-1,i,-i\}$, 因为它们的乘积可以得到 1。
在 $\mathbb{Z}[i]$ 里, 如果一个数乘以另一个数等于 1, 那么这个数就是 unit.
Def: z, z'be gaussian integers, we say z divides z' if there exists a gaussian integer w such that $z'=zw$.
在 $\mathbb{Z}[i]$ 里, 如果 z 乘以一个数等于 z', 那么我们说 z divides z'.
irreducible:
Def: A gaussian integer z is called irreducible if it is not a unit and cannot be expressed as a product of two non-unit gaussian integers.
在 $\mathbb{Z}[i]$ 里, 如果一个数不是 unit, 且不能写成两个非 unit 的积, 那么这个数就是 irreducible.
$z$irreducible ⟷ z不能分解成两个 non-units 的乘积
eg. 2 is irreducible as an integer, but it is not irreducible as a gaussian integer since $2=(1+i)(1-i)$, they are not units.
Prime
Def: let $z\neq 0$ and not unit, if $z\mid uv\longleftrightarrow z\mid u$ or $z\mid v$, then z is called a prime in $\mathbb{Z}[i]$.
在 $\mathbb{Z}[i]$ 里, 如果一个数满足: 如果它能整除一个积, 那么它至少能整除其中的一个因子, 那么这个数就是 prime.
Division algorithm for gaussian integers:
Theorem: let $z, w\in\mathbb{Z}[i]$ with $w\neq 0$, then there exist unique $q, r\in\mathbb{Z}[i]$ with $z=qw+r$ and $|r|<|w|$.
在 $\mathbb{Z}[i]$ 里, 对于任意两个数 z, w (其中 $w\neq 0$), 都可以找到唯一的 q, r 使得 $z=qw+r$ 且 $|r|<|w|$.
gcd in $\mathbb{Z}[i]$:
Def: Let $z, w\in\mathbb{Z}[i]$ with $w\neq 0$, a gcd of z and w is a gaussian integer d such that: $d\mid z$ and $d\mid w$; And for any gaussian integer c with $c\mid z$ and $c\mid w$, we have $c\mid d$.
Bezout lemma for Gaussian integers:
Def: For any two gaussian integers a and b, there exist gaussian integers s and t such that $as+bt=\gcd(a,b)$.
Prime iff irreducible in $\mathbb{Z}[i]$:
Theorem: A gaussian integer z, $z=a+bi$ is prime if and only if it is irreducible.
证明 prime ⟹ irreducible: if z is prime, then $z\mid uv\longleftrightarrow z\mid u$ or $z\mid v$. if $z\mid u$, then $u=zw$ for some gaussian integer w. if $z\mid v$, then $v=zw'$ for some gaussian integer w'. 代回 $z=uv=(zw)v$, 得到 $1=uv$. 所以 v 是 unit, z 是 irreducible
证明 irreducible ⟹ prime: if z is irreducible, $z\mid uv$, and if $z\nmid w\longrightarrow gcd(z,w)$ is unit, sp gcd(z,w)=1. By gaussion Beozut, $1=sz+tw\longrightarrow v=szv+twv$. since $z\mid zv$ and $z\mid wv$, $z\mid v$ so z is prime.
判断一个 gaussian integer 是否 prime, 先判断它是否 irreducible.
eg. $2=(1+i)(1-i)$,is not units, and it is irreducible, so 2 is not prime in $\mathbb{Z}[i]$. $7=(a+bi)(c+di)$, 两边平方, $(a^2+b^2)=(c^2+d^2)=7$,not possible, so 7 is irreducible, so 7 is prime in $\mathbb{Z}[i]$.
那些整数可以被写成平方和?
Theorem 1: Let $p\equiv 4k+1$/ $p\equiv 1 mod 4$ be a prime in $\mathbb{Z}$ for some integer $k>0$. Then $p=a^2+b^2$ for some integers a and b. (3.7.11 不能被写成两个整数平方和) (只针对 p 的定理)
Theorem 2: let $1<n\in\mathbb{N}, n=2^cp_1^{\alpha_1}\cdots p_k^{\alpha_k}q_1^{d_1}\cdots q_t^{d_t}$, where

$p_i \equiv 1 \pmod 4$and$q_j \equiv 3 \pmod 4$.

Associate

Def: Two gaussian integers z is an associate of w if there exists a unit u such that $z = uw$.

if z is an associate of w, the w is an associate of z.

Fundamental Theorem of Arithmetic for Gaussian integers:

Theorem: Every nonzero gaussian integer that is not a unit can be factored into a product of primes in $\mathbb{Z}[i]$. Moreover, this factorization is unique up to the order of the factors and multiplication by units.

在 $\mathbb{Z}[i]$ 里, 除了是 unit 的 integer 之外, 每一个非零的非 unit 的 gaussian integer 都可以写成 prime 的积, 而且这个分解是唯一的, 除了因子顺序和乘以 units 之外.

Monte Carlo Approximation:

If $x_1, \dots, x_n \overset{i.i.d.}{\sim} p(x \mid \theta)$, then by SLLN $\mathbb{E}[f(x) \mid \theta] \approx \frac{1}{n} \sum_{i=1}^n f(x_i)$.

Inverse-CDF Sampling: 若目标分布的 CDF 为 F, 可先取 $U \sim \text{Unif}(0,1)$, 令 $U = F(X)$; $X = F^{-1}(U)$; now $P(X \le x) = F(x)$, so X 服从目标分布。

CDF Pseudo-Inverse: CDF 只要求单调不减, 未必是一一对应; 若 F 不可直接求逆, 找"最小的使累计概率达到 u 的 x"。$F^{-1}(u) = \inf\{x : \ F(x) \ge u\}$,

Examples: 若 $X \sim \text{Exp}(\lambda), F(x) = 1 - e^{-\lambda x}$, 则 $X = -\frac{1}{\lambda} \log(1-U)$. 若 $X \sim \text{Bern}(\theta)$, 则 $X = \mathbf{1}\{U > 1-\theta\}$.

Transformation Formula: If $Y = f(X)$ and the solutions to $f(x) = y$ are $x_1, \dots, x_K$, then $p_Y(y) = \sum_{k=1}^K \frac{p_X(x_k)}{|f'(x_k)|}$.

Examples: If $Y = aX + b$, then $p_Y(y) = \frac{1}{|a|} p_X\left(\frac{y-b}{a}\right)$. If $Y = X^2$, then $p_Y(y) = \frac{1}{2\sqrt{y}} \left(p_X(\sqrt{y}) + p_X(-\sqrt{y})\right)$.

Gamma Sampling for Integer Shape: 若 $Y \sim \text{Gamma}(a, b)$ 且 a 为正整数, 则 $Y = \sum_{i=1}^a X_i$, 其中 $X_1, \dots, X_a \overset{i.i.d.}{\sim} \text{Exp}(b)$. 直觉上: Gamma$(a, b)$ 是"等到第 a 次事件发生的总等待时间"。

Rejection Sampling Setup: 想从目标分布 $p(z)$ 中采样, 但不能直接计算, 只知道它和某个未归一化函数 $\tilde{p}(z)$ 成正比 $p(z) = \frac{1}{M}\tilde{p}(z)$, where M is unknown. Choose proposal $q(z)$ and constant k such that $kq(z) \ge \tilde{p}(z)$ for all z, with $\text{supp}(p) \subset \text{supp}(q)$.

Rejection Sampling Algorithm: 1. Sample $z \sim q(\cdot)$. 2. Sample $u \sim \text{Unif}[0, kq(z)]$. 3. Accept z if $u \le \tilde{p}(z)$.

Acceptance Probability: $P(\text{accept}) = \int \frac{\tilde{p}(z)}{kq(z)} q(z)\, dz = \frac{M}{k}$. Hence we want k as small as possible subject to $kq(z) \ge \tilde{p}(z)$.

Why Rejection Sampling Works: Accepted samples have cdf $P(z \le z_0 \mid \text{accepted}) = \frac{1}{M} \int_{z \le z_0} \tilde{p}(z)\, dz$, so accepted z is distributed according to $p(z)$.

Rejection Sampling for Bayesian Inference: For posterior $p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta) p(\theta)$, take $\tilde{p}(\theta) = p(\mathcal{D} \mid \theta) p(\theta)$. If choose proposal $q(\theta) = p(\theta)$, then $k = \max_\theta \frac{\tilde{p}(\theta)}{q(\theta)} = \max_\theta p(\mathcal{D} \mid \theta)$, i.e. the maximum likelihood value. 先从 prior 采样作为 proposal, 再用 likelihood 决定是否 accept; likelihood 越大, 越容易被 accept

Importance Sampling:

不直接从 $p(z)$ 采样, 而是从一个容易采样的 proposal distribution $q(z)$ 采样。用 weight 修正 $q(z)$ 和 $p(z)$ 之间的差异。

因为: $\mathbb{E}_p[f(z)] = \int f(z) p(z)\, dz, = \int f(z) \frac{p(z)}{q(z)} q(z)\, dz. = \mathbb{E}_q\left[f(z) \frac{p(z)}{q(z)}\right]$.

定义 importance weight: $w(z) = \frac{p(z)}{q(z)}$.

于是如果 $z_1, \dots, z_n \sim q(z)$, 则有: $\mathbb{E}_p[f(z)] \approx \frac{1}{n} \sum_{i=1}^n w(z_i) f(z_i)$,

直观理解: 从 q 采样, 用 $\frac{p}{q}$ 修正采样偏差。

如果某个区域 $p(z)$ 大但 $q(z)$ 小, 说明这个区域在 target distribution 中很重要, 但 proposal distribution 很少采到, 所以采到后要给更大的 weight。

Support Condition:

Importance Sampling 要求 proposal distribution $q(z)$ 覆盖 target distribution $p(z)$ 重要的区域。$\text{supp}(p) \subseteq \text{supp}(q)$.

如果我们要估计: $\mathbb{E}_p[f(z)]$, 则需要: $\text{supp}(f(z)p(z)) \subseteq \text{supp}(q)$.

只要某个地方: $f(z)p(z) \ne 0$, 就必须有: $q(z) > 0$.

否则, 如果某个区域 $p(z) > 0$ 但 $q(z) = 0$, 那么从 $q(z)$ 永远采不到这个区域

Self-normalized Importance Sampling:

有时 target density $p(z)$ 不是直接已知的, 而是只知道一个 unnormalized density: $\tilde{p}(z)$.,

$p(z) = \frac{\tilde{p}(z)}{Z}$, 其中: $Z = \int \tilde{p}(z)\, dz$ 是 normalization constant。

如果 Z 不知道, 就不能直接计算: $w(z) = \frac{p(z)}{q(z)} = \frac{\tilde{p}(z)}{Z q(z)}$., Z unknown。

因此改用 unnormalized importance weight: $\tilde{w}(z) = \frac{\tilde{p}(z)}{q(z)}$.

然后用 Self-normalized Importance Sampling estimator: $\mathbb{E}_p[f(z)] \approx \frac{\sum_{i=1}^n \tilde{w}(z_i) f(z_i)}{\sum_{i=1}^n \tilde{w}(z_i)}$, $z_i \sim q(z)$.

Rare-event Probability Estimation: 想估计一个 p(rare-event) 可以选择一个 proposal distribution q, 让它更容易采到 tail region (a, ∞), 提高估计效率。

eg:$P(X > a)$. 如果直接从 p 采样, 可能很少采到 $X > a$ 的样本, 导致 estimator variance 很大。

因为: $P(X > a) = \mathbb{E}_p\left[\mathbf{1}_{\{X > a\}}\right] = \int \mathbf{1}_{\{z > a\}} p(z)\, dz$.

使用 proposal distribution $q(z)$ 后: $P(X > a) = \int \mathbf{1}_{\{z > a\}} \frac{p(z)}{q(z)} q(z)\, dz \approx \frac{1}{n} \sum_{i=1}^n \mathbf{1}_{\{z_i > a\}} \frac{p(z_i)}{q(z_i)}$, $z_i \sim q(z)$. q 的取值覆盖 >a 那段 tail region 就可以

Sampling Importance Resampling (SIR):

Importance Sampling 得到的是 weighted samples: $(z_i, \tilde{w}_i), i = 1, \dots, n$.

SIR 的作用是: 用 normalized weights $\bar{w}_i$ 来 resampling

步骤如下:

1.Sample proposal samples $z_1, \dots, z_n \sim q(z)$, $q(z)$ 易采样

2.Compute unnormalized importance weights for each sample: $\tilde{w}_i = \frac{\tilde{p}(z_i)}{q(z_i)}$

3.Normalize the weights, 使和 =1: $\bar{w}_i = \frac{\tilde{w}_i}{\sum_{j=1}^n \tilde{w}_j}$, 因此 $\sum_{i=1}^n \bar{w}_i = 1$, $\bar{w}_i$ 可以作 resampling probabilities.

4.Resample with replacement from $\{z_1, \dots, z_n\}$ according to probabilities $\{\bar{w}_1, \dots, \bar{w}_n\}$, 得到 resampled points $z_1^*, \dots, z_m^*$。Resampling 后得到的样本近似服从 target distribution $p(z)$。当 $n \to \infty$ 时,resampled points 的 empirical distribution 会收敛到 $p(z)$。

SIR for Bayesian Inference: $p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta) p(\theta)$.

因此 unnormalized posterior density 是: $\tilde{p}(\theta) = p(\mathcal{D} \mid \theta) p(\theta)$.

如果选择 prior distribution 作为 proposal distribution: $q(\theta) = p(\theta)$, 那么 importance weight 为: $\tilde{w}(\theta_i) = \frac{\tilde{p}(\theta_i)}{q(\theta_i)} = \frac{p(\mathcal{D} \mid \theta_i) p(\theta_i)}{p(\theta_i)} = p(\mathcal{D} \mid \theta_i)$.

Normalized weights 为: $\bar{w}_i = \frac{p(\mathcal{D} \mid \theta_i)}{\sum_{j=1}^n p(\mathcal{D} \mid \theta_j)}$.

SIR for Bayesian inference 的步骤是: 1.Sample $\theta_1, \dots, \theta_n \sim p(\theta)$. 2.Compute weights: $\tilde{w}_i = p(\mathcal{D} \mid \theta_i)$. 3.Normalize the weights. 4.Resample θ_i according to the normalized weights.

Resampled θ values 可以看作 approximate samples from posterior distribution: $p(\theta \mid \mathcal{D})$.

Monte Carlo EM:

在 EM algorithm 中,E-step 需要计算: $Q(\theta \mid \theta^{(m)}) = \mathbb{E}_{p(z \mid x, \theta^{(m)})}[\log p(x, z \mid \theta)] = \int p(z \mid x, \theta^{(m)}) \log p(x, z \mid \theta)\, dz$.

若积分无法解析计算, 用 Monte Carlo approximation。从 latent variable posterior 中采样: $z_1, \dots, z_n \sim p(z \mid x, \theta^{(m)})$.

然后: $Q(\theta \mid \theta^{(m)}) \approx \frac{1}{n} \sum_{i=1}^n \log p(x, z_i \mid \theta)$.

如果我们能直接从 $p(z \mid x, \theta^{(m)})$ 采样, 那么这是普通 Monte Carlo approximation。如果不能直接采样, 也可以引入 proposal distribution $q(z)$, 再用 Importance Sampling 近似这个 E-step expectation。

MCMC Goal: 希望从高维的 target pdf $\pi(x)$ 中进行采样。构造一个 Markov chain $Z_0, Z_1, \dots,$ 使得当 $n \to \infty$ 时,Z_n 的分布趋近于 $\pi(\cdot)$。

Stationary Distribution: 对于一个具有 transition probability $T(x, y)$ 的 homogeneous Markov chain, 若 $\sum_x \pi(x) T(x, y) = \pi(y)$ 对所有 y 都成立, 则 π 是 stationary distribution。Matrix form: $\pi T = \pi$.

Asymptotic Convergence: 如果该 chain 是 irreducible、positive recurrent 且 aperiodic, 那么 $\pi_k = \pi_0 T^k \to \pi$ 当 $k \to \infty$ 时成立, 其中 π 是 stationary distribution。这样的 chain 是 ergodic。(遍历的)

Reversibility: A sufficient condition for π to be stationary is $\pi(x)T(x, y) = \pi(y)T(y, x) \forall x, y.$ 在平稳状态下, 从 x 到 y 的概率流量, 等于从 y 到 x 的概率流量

Burn-In: Discard the initial part of the chain before it is close to stationarity. Typical burn-in: first 1000 to 5000 samples.

Thinning: MCMC samples are usually correlated, especially consecutive samples. Thinning is used to reduce autocorrelation among the retained samples, making them more nearly independent.

Metropolis–Hastings (MH): 假设 target 只知道到 normalization 常数为止: $\pi(x) \propto \tilde{\pi}(x)$. 选择 proposal $q(x, y)$。在第 m 步: 当前状态为 $x = Z_{m-1}$, 提出 proposal $y \sim q(x, \cdot)$, 以如下 accept probability 接受 $A(x, y) = \min\left(1, \frac{\tilde{\pi}(y)}{\tilde{\pi}(x)} \frac{q(y, x)}{q(x, y)}\right)$. 若被接受, 则令 $Z_m = y$; 否则令 $Z_m = x$。

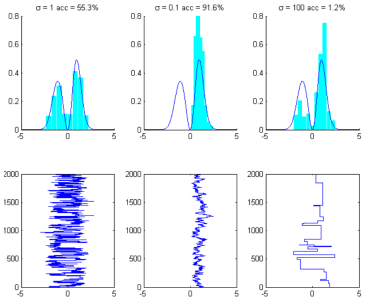
Why MH Works: The MH chain is reversible with respect to π, so π is a stationary distribution. If the chain is also irreducible and aperiodic, then Z_n approaches π in distribution. MH 先保证"目标分布是平稳的", 再加上链本身"能到处走且不周期卡住", 就能保证长期采样结果逼近目标分布。

Random Walk MH: 从当前点 x 加一个随机扰动得到候选点 y。Choose proposal of the form $q(x, y) = q(y - x)$. Common choices: $y - x \sim \mathcal{N}(0, \Sigma)$ or $y - x \sim \text{Unif}[-\delta, \delta]^d$. acceptance rate about 0.25 to 0.5

Metropolis Algorithm (Symmetric Proposal): 如果提议分布是对称的, 即 $q(x, y) = q(y, x)$, then acceptance simplifies to $A(x, y) = \min\left(1, \frac{\tilde{\pi}(y)}{\tilde{\pi}(x)}\right)$. 候选点密度更高就必接收, 更低就按比例概率接收

Independence Chain MH: Choose proposal independent of current state: 每一步候选点 y 都直接从同一个分布 q 抽, 不管现在在 x 的哪里。$q(x, y) = q(y)$. Works well only if q approximates π well, usually with heavier tails than π. 这样能更稳地覆盖 π 的尾部, 避免某些区域几乎提议不到, 导致混合变差或接受率很低。

acceptance rate close to 1 is preferred. ***Proposal Tuning***: Random walk MH:



too small variance: slow exploration, high acceptance rate. too large variance:big jumps, low acceptance rate, chain gets stuck.

Gibbs Sampling: 想从多变量分布 $p(z_1, \dots, z_d)$ 中采样. If full conditionals $p(z_i \mid z_{-i})$ are available, iterate: $z_1^{(k)} \sim p(z_1 \mid z_2^{(k-1)}, \dots, z_d^{(k-1)})$, $z_2^{(k)} \sim p(z_2 \mid z_1^{(k)}, z_3^{(k-1)}, \dots, z_d^{(k-1)})$, $z_d^{(k)} \sim p(z_d \mid z_1^{(k)}, \dots, z_{d-1}^{(k)})$.

Why Gibbs Works: Each Gibbs update leaves the target joint distribution invariant. Equivalently, Gibbs is an MH step with acceptance probability 1.

How to Get Full Conditionals: Use $p(z_i \mid z_{-i}) = \frac{p(z_1, \dots, z_d)}{p(z_{-i})}$. In practice: start from the joint density, ignore all terms not depending on z_i, and identify the resulting distribution.

Gibbs Example:

1. For $p(x, y, n) \propto \binom{n}{x} y^{x+a-1} (1-y)^{n-x+b-1} e^{-\lambda} \frac{\lambda^n}{n!}$, the full conditionals are $x \mid y, n \sim \text{Bin}(n, y)$, $y \mid x, n \sim \text{Beta}(x + a, n - x + b)$, $n \mid x, y \sim \text{Pois}((1-y)\lambda) + x$.

2. $p(z_1, z_2, z_3) \propto z_1^{z_2+a-2} (1-z_1)^{b-1} \frac{1}{z_2!} \mathbf{1}_{\{z_3 \in [0, z_1]\}}$.

$p(z_1 \mid z_2, z_3) \propto z_1^{z_2+a-2} (1-z_1)^{b-1} \mathbf{1}_{\{z_1 \ge z_3\}}$, so $z_1 \mid z_2, z_3 \sim \text{Beta}(z_2 + a - 1, b)$ truncated to $[z_3, 1]$.

$p(z_2 \mid z_1, z_3) \propto \frac{z_1^{z_2}}{z_2!}$, so $z_2 \mid z_1, z_3 \sim \text{Pois}(z_1)$.

$p(z_3 \mid z_1, z_2) \propto \mathbf{1}_{\{0 \le z_3 \le z_1\}}$, so $z_3 \mid z_1, z_2 \sim \text{Unif}(0, z_1)$.

Initialize $(z_1^{(0)}, z_2^{(0)}, z_3^{(0)})$. For each iteration k, update: $z_1^{(k)}, z_2^{(k)}, z_3^{(k)}$

Poisson distribution is $\mathbb{P}(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}$,

Ways to Obtain Approximately i.i.d. Samples from Gibbs: 因为 Gibbs 连续产生的样本通常有相关性, 不是严格独立, 所以用两种常见策略减弱相关性: Option 1: run r independent chains and keep the final sample from each. Option 2: run one long chain, discard burn-in, then thin by keeping every dth sample.

Gibbs for Binary Hidden States: Gibbs sampling 用于从 posterior $p(z \mid x)$ 中采样 hidden states。每次固定其他 variables, 只更新一个 z_i: $z_i \sim p(z_i \mid z_{-i}, x)$.

Full conditional: $p(z_i \mid z_{-i}, x) \propto$ terms involving z_i. 实际操作: 从 joint distribution 中只保留和 z_i 有关的 terms, 其他 terms 当作 constants 忽略。

If $z_i \in \{D, U\}$, compute two unnormalized weights: $w_D = P(z_i = D) P(x_i \mid z_i = D) \psi(D, z_{i-1}, z_{i+1})$, $w_U = P(z_i = U) P(x_i \mid z_i = U) \psi(U, z_{i-1}, z_{i+1})$.

Then normalize: $P(z_i = D \mid z_{-i}, x) = \frac{w_D}{w_D + w_U}$, $P(z_i = U \mid z_{-i}, x) = \frac{w_U}{w_D + w_U}$.

Artificial Neuron: Local field / pre-activation: $v = \sum_{j=1}^m w_j x_j + b$. Bias as an additional input: $v = \sum_{j=0}^m w_j x_j$, with $x_0 = 1$ and $w_0 = b$.

Neuron output: $y = f(v)$.

Common Activation Functions: Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$. Threshold / step function: $\sigma(v) = 1$ if $v \ge 0$, and 0 otherwise. ReLU: $\text{ReLU}(x) = \max(0, x)$.

Single-Neuron Perceptron: Decision rule: $y = 1$ if $w^\top p + b > 0$, otherwise $y = 0$. Decision boundary: $w^\top p + b = 0$. The weight vector w is orthogonal to the boundary. Positive decision region: $w^\top p + b > 0$.

Decision Boundary Geometry: All points on the boundary satisfy the same inner product: $w^\top p = -b$. Scaling by any nonzero constant does not change the boundary: $w^\top p + b = 0 \iff c\, w^\top p + c\, b = 0$.

OR Perceptron Example: A valid choice is $w = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $b = -0.5$. Then $w^\top p + b = p_1 + p_2 - 0.5$, which implements OR with a threshold neuron.

AND Perceptron Example: A valid choice is $w = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $b = -1.5$. Then $w^\top p + b = p_1 + p_2 - 1.5$, which implements AND with a threshold neuron.

Perceptron Limitation: A single-layer perceptron can represent only a linear decision boundary, so it cannot solve linearly inseparable problems such as XOR.

XOR with a Multi-Layer Network: One construction uses two hidden neurons: $h_1 = \sigma\left(\begin{bmatrix} 1 & 1 \end{bmatrix} p - 0.5\right)$, $h_2 = \sigma\left(\begin{bmatrix} -1 & -1 \end{bmatrix} p + 1.5\right)$. Output neuron: $y = \sigma(h_1 + h_2 - 1.5)$.

Multi-Layer Perceptron (MLP) Forward Pass: For a 3-layer network, $h_1 = \sigma(W_1 p + b_1)$, $h_2 = \sigma(W_2 h_1 + b_2)$, $\hat{y} = \sigma(W_3 h_2 + b_3)$. Dimension reminder: if $h_1 \in \mathbb{R}^4$, $h_2 \in \mathbb{R}^2$, $\hat{y} \in \mathbb{R}$, then $W_1 \in \mathbb{R}^{4 \times d}$, $W_2 \in \mathbb{R}^{2 \times 4}$, $W_3 \in \mathbb{R}^{1 \times 2}$.

Universal Approximation / Decision Regions: A 1-layer network gives a linear decision boundary. A 2-layer network can generate a single convex decision region. A 3-layer MLP can approximate arbitrary decision regions with enough hidden neurons.

Universal Approximation Theorem (informal): A 2-layer MLP with nonconstant, bounded, and continuous hidden neurons, plus a linear output neuron, can approximate any continuous function arbitrarily well if it has enough hidden neurons.

Forward Propagation: For one layer, $a^{(l)} = \sigma\left(W^{(l)} a^{(l-1)} + b^{(l)}\right)$. With input $a^{(0)} = x$, an L-layer MLP computes $y = f(x) = a^{(L)} = \sigma\left(W^{(L)} \sigma\left(W^{(L-1)} \cdots \sigma\left(W^{(1)} x + b^{(1)}\right) \cdots + b^{(L-1)}\right) + b^{(L)}\right)$.

Network Parameters: $\theta = \{W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}, \dots, W^{(L)}, b^{(L)}\}$.

Softmax Output Layer: For logits $z = (z_1, \dots, z_K)$, $\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$, $0 < \hat{y}_k < 1$, $\sum_{k=1}^K \hat{y}_k = 1$.

Common Activation Functions: Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$. Tanh: $\tanh(x) = 2\sigma(2x) - 1$. ReLU: $\text{ReLU}(x) = \max(0, x)$. Leaky ReLU: $\text{LReLU}(x) = \begin{cases} \alpha x, & x < 0, \\ x, & x \ge 0. \end{cases}$ Linear / identity: $f(x) = cx$ (often $c = 1$).used in regression output layer. Also known as identity activation function.

Data Preprocessing: Standardization: $x_{\text{std}} = \frac{x - \mu}{\sigma}$. Min-max normalization: $x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$.

Classification Targets: For K classes, use K output neurons and one-hot targets: $y = (0, \dots, 0, 1, 0, \dots, 0)$. Prediction: $\hat{c} = \arg\max_k \hat{y}_k$.

Training Objective: For dataset $\{(x^{(i)}, y^{(i)})\}_{i=1}^N$, $\mathcal{L}(\theta) = \sum_{i=1}^N \mathcal{L}_i(\theta)$, or often the average loss $\frac{1}{N} \sum_{i=1}^N \mathcal{L}_i(\theta)$.

Cross-Entropy Loss: For multi-class classification, $\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_k^{(i)} \log \hat{y}_k^{(i)}$. 其中 N 是 samples 数量,K 是 classes 数量,$y_k^{(i)}$ is true label,$\hat{y}_k^{(i)}$ 是 predicted probability。

Binary form used in the slides: $\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K \left(y_k^{(i)} \log \hat{y}_k^{(i)} + (1 - y_k^{(i)}) \log(1 - \hat{y}_k^{(i)})\right)$. 对于 binary classification,$y \in \{0, 1\}$,$\hat{y} = P(y = 1 \mid x)$。若 $y = 1$,loss 变成 $-\log \hat{y}$; 若 $y = 0$,loss 变成 $-\log(1 - \hat{y})$。

Mean Squared Error (MSE): For regression, $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2$. Mean absolute error (MAE): $\frac{1}{n} \sum_i |y_i - \hat{y}_i|$

Gradient Descent (GD): Gradient: $\nabla \mathcal{L}(\theta) = \left[\frac{\partial \mathcal{L}}{\partial \theta_i}\right]_i$. Update rule: $\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla \mathcal{L}(\theta_{\text{old}})$, where α is the learning rate.

Backpropagation: Each training step uses: forward pass $\to$ compute loss $\to$ backward pass $\to$ update parameters. Backprop computes $\nabla \mathcal{L}(\theta)$ using the chain rule.

Mini-Batch Gradient Descent: For a mini-batch $\mathcal{B}$, $\mathcal{L}_{\mathcal{B}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathcal{L}_i(\theta)$, and update using $\nabla \mathcal{L}_{\mathcal{B}}(\theta)$. Typical batch size: 32 to 256.

Stochastic Gradient Descent (SGD): SGD uses a mini-batch of size 1. In practice, "SGD"in DL libraries usually means mini-batch GD, often with momentum.

Gradient Descent with Momentum: Use running velocity v_t: $v_t = \beta v_{t-1} - \alpha \nabla \mathcal{L}(\theta_{t-1})$, $\theta_t = \theta_{t-1} + v_t$.

Nesterov Momentum: Compute the gradient at a look-ahead point: $v_t = \beta v_{t-1} - \alpha \nabla \mathcal{L}(\theta_{t-1} + \beta v_{t-1})$, $\theta_t = \theta_{t-1} + v_t$.

Adam: Adam uses moving averages of gradients and squared gradients: $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$, $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$. Commonly used together with bias correction and adaptive updates.

Learning Rate: α controls the step size of parameter updates. Too small $\Rightarrow$ very slow convergence. Too large $\Rightarrow$ unstable training, divergence, saturated activations, or dead ReLU units.

Learning Rate Scheduling: Annealing / decay reduces α over time to make training more stable. Warmup gradually increases α at the beginning, then decays it later. Common schedules: step decay, exponential decay, cosine decay, reduce-on-plateau.

Vanishing / Exploding Gradients: Very small gradients cause weights to update very slowly or stop updating. Very large gradients cause unstable updates or divergence. This is common in very deep networks due to repeated chain-rule multiplication. Common remedies: proper initialization, ReLU-based activations, Batch Normalization, residual connections, and gradient clipping.

Saturated Activations: Sigmoid and tanh can saturate when inputs are very large or very small, causing gradients to become close to zero. This leads to slow learning or weights stopping updates. Common remedies: Xavier initialization, Batch Normalization, input normalization, and using ReLU-based activations.

Dead ReLU: A ReLU unit is dead if its input is always negative, so its output and gradient are always zero. The weights connected to this unit may stop updating. Common remedies: smaller learning rate, He initialization, Leaky ReLU, ELU, or Batch Normalization.

Generalization: Underfitting: high training error and high validation error. Overfitting: low training error but high validation error.

Model Capacity: More neurons / more layers $\Rightarrow$ higher capacity. Too little capacity $\Rightarrow$ underfitting. Too much capacity $\Rightarrow$ higher overfitting risk and harder optimization.

ℓ_2 Regularization: $\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \lambda \sum_k \theta_k^2$.

ℓ_1 Regularization: $\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \lambda \sum_k |\theta_k|$.

Elastic Net: $\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \lambda_1 \sum_k |\theta_k| + \lambda_2 \sum_k \theta_k^2$.

Dropout: Randomly drop units during training to reduce overfitting. Let p be the probability that a unit is retained: $\tilde{h}_i = m_i h_i$, $m_i \sim \text{Bernoulli}(p)$. Disable dropout during evaluation and use all units, scaling by p: $\tilde{h}_i = p h_i$. Equivalently, with inverted dropout, use $\tilde{h}_i = \frac{m_i h_i}{p}$ during training and $\tilde{h}_i = h_i$ during evaluation.

Early Stopping: Use a validation set and stop training when validation loss /

accuracy does not improve for n epochs. The hyper-parameter n is called *patience*.

Batch Normalization: For a batch input x with mean μ and standard deviation σ, $\hat{x} = \frac{x-\mu}{\sigma}$(标准化) $x_{\text{BN}} = \gamma\hat{x} + \beta$.(把标准化后的值进行缩放和平移) Here, γ and β are learnable scale and shift parameters. In CNNs, BatchNorm usually has $2C$ learnable parameters for C channels. It does not change the activation shape.
Benefits: stabilizes training, improves gradient flow, allows larger learning rates, and reduces sensitivity to initialization.

Hyper-Parameter Tuning: • layer size+num neur, LR(schedule), optimizer type, regularization(L2), batch size, activation/loss function.
• Methods: grid search(check all with step), random search(prefer), Bayesian optimization. Use k-fold cross-validation if data is limited.

k-Fold Cross-Validation: Split training data into k folds. Train on $k-1$ folds, validate on the remaining fold, repeat for all folds, then average validation results. Used mainly when training data is limited.

Deep vs. Shallow Networks: Deeper networks often perform better than shallow ones, but only up to a limit; performance may plateau after too many layers.

Residual Networks (ResNets)
• Use identity=skip connections(out=in +sigma) • prevent vanishing gradients.
• very deep models(18, 50, 152 layers).

From MLP to CNN: MLPs are not ideal for images because they flatten the image into a vector, which ignores the original spatial structure and makes the model sensitive to object position. The same pattern shifted to a different location may produce a very different output. CNNs are more suitable for image tasks because they use local connectivity to extract local features such as edges, corners, and strokes, and use weight sharing to apply the same filters across different positions. This preserves spatial structure, reduces the number of parameters, and makes CNNs more efficient and less prone to overfitting.

Convolution:
1D Continuous form: $(x * w)(t) = \int_{-\infty}^{\infty} x(\tau)\, w(t-\tau)\, d\tau$
Discrete form: $(x * w)[n] = \sum_{k=-\infty}^{\infty} x[k]\, w[n-k]$
egample: $x = [1, 2, 3, 4]$, $w = [5, 6, 7]$, 卷积后得到
$z = [5, 16, 34, 52, 45, 28]$
2D discrete form:
$(x * w)[m, n] = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} x[i, j]\, w[m-i,\ n-j]$

Local receptive field:In CNNs, neurons in a layer are only connected to a small spatial region of the previous layer. ***Pooling***:
Max pooling and average pooling.Pooling layers reduce the spatial size of the feature maps. Reduce the number of parameters, prevent overfitting.

CNNs learn hierarchical features: early layers detect edges and textures, middle layers detect parts and patterns, and deeper layers capture semantic features for final classification.

Convolution Output Size and Zero Padding:

$$N_{\text{out}} = \frac{N-F}{S} + 1,\ N_{\text{out,pad}} = \frac{N+2P-F}{S} + 1$$

For odd filter size F and stride $S = 1$, choose $P = \frac{F-1}{2}$ so that $N_{\text{out,pad}} = N$. Without padding, repeated convolutions shrink spatial size (e.g. $32 \to 28 \to 24$)Shrinking too fast is not good.

Example:
Input: $32 \times 32 \times 3,\ K = 10,\ F = 5,\ S = 1,\ P = 2$

$$N_{\text{out}} = \frac{N+2P-F}{S} + 1 = \frac{32 + 2 \cdot 2 - 5}{1} + 1 = 32$$

Output volume: $32 \times 32 \times 10$
Params/filter: $F \cdot F \cdot C_{\text{in}} + 1 = 5 \cdot 5 \cdot 3 + 1 = 76$
Total params: $K \cdot 76 = 10 \cdot 76 = 760$

Code Example:
```
conv1 = torch.nn.Conv2d(in_channels=3, out_channels=2,
kernel_size=3, stride=1, padding=0)
print(conv1(x.unsqueeze(0)).shape) # add batch dimension
```

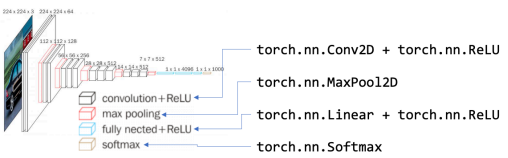
Pooling layer:
1. Makes the representations smaller and more manageable.
2. Operates over each activation map independently.
Assume input is W1 x H1 x C, Conv layer needs 2 hyperparameters:
1. The spatial extent (filter size) F,pooling window 的大小
2. The stride S
Output W2 x H2 x C:
W2 = (W1 - F)/S + 1, H2 = (H1 - F)/S + 1
Number of parameters: 0
```
pool1 = torch.nn.MaxPool2d(kernel_size=2, stride=2,
padding=0)
# Commonly, stride == kernel_size
```

Flatten: Flatten only changes shape, not values. It converts spatial feature maps into vectors for FC/Linear layers.

Layers with no parameters: Input, activation layers such as ReLU, Leaky ReLU, sigmoid and tanh, pooling layers, dropout, flatten, and softmax layers have no learnable parameters. They may have hyperparameters, such as pooling size, stride, or dropout probability, but these are not learned during training.

Flatten example:
Input volume: $8 \times 8 \times 16$ Flatten output size: $8 \cdot 8 \cdot 16 = 1024$
Output: 1024 Total params: 0

Fully-connected layer example:
Input size: 1024, number of output neurons: 10
Weights: $1024 \cdot 10$ Biases: 10
Total params: $1024 \cdot 10 + 10 = 10250$ Output: 10

Implementation:


Architecture pattern:
[(CONV-RELU)* N - POOL?]* M - (FC-RELU) * K - SOFTMAX
```
model = nn.Sequential(
nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3,
stride=1, padding=1),
nn.ReLU(),
nn.MaxPool2d(kernel_size=2, stride=2),
nn.Flatten(),
nn.Linear(in_features=..., out_features=64),
nn.ReLU(),
nn.Linear(in_features=64, out_features=num_classes),
nn.Softmax(dim=1))
```

Gradient Descent (GD)

1. **Initialization:** Randomly initialize the model parameters θ^0. Set $\theta^{\text{old}} = \theta^0$.
2. Compute the gradient of the loss function at θ^{old}: $\nabla\mathcal{L}(\theta^{\text{old}})$.
3. Update the parameters: $\theta^{\text{new}} = \theta^{\text{old}} - \alpha\nabla\mathcal{L}(\theta^{\text{old}})$ where α is the learning rate.
4. repeat until terminating.
GD does not guarantee reaching a global minimum.
• Backpropagation: Combines forward pass + backward pass using chain rule to compute gradients of loss w.r.t. weights.
• Batch GD: compute gradients over full dataset.
• Mini-batch GD: over subset (32 to 256 samples). Mini-batch gradient approximates the full training set gradient well.
• SGD: mini-batch size = 1, fast but noisy. Less used.

GD with Momentum:
• Momentum: $v_t = \beta v_{t-1} + (1-\beta)\nabla\mathcal{L}$, improves convergence.
• Nesterov Momentum: lookahead at next step before computing gradient. (Skip)
• Adam: computes a weighted average of past gradients(1st moment of G) and weighted average of past squared gradients(2rd). • Other: RMSprop, Adagrad, Adadelta, Nadam, etc. • Most used: Adam, SGD with momentum.